Project Assignment: Robot Model Telemetry Refactor

This assignment is for an advanced student who will plan and execute a full refactor of telemetry handling in the Robot Model.

The current design pushes telemetry through the hierarchy as a method parameter and also lets several concrete classes write directly to telemetry on their own. That split ownership has already created inconsistent behavior, dead telemetry hooks, and autonomous code that bypasses the object model.

Your job is to replace that pattern with a single, consistent telemetry architecture.

Assignment Goal

Refactor the Robot Model so that the single telemetry instance created by the `OpMode` is injected into each abstract layer of the model and owned there, instead of being passed down every loop through `updateTelemetry(Telemetry telemetry)` methods.

The target architecture must also give the autonomous-facing layers their own telemetry write methods so autonomous code can report state without taking raw `Telemetry` parameters everywhere.

This assignment includes both design work and implementation work.

Core Requirements

Your refactor must satisfy all of the following:

1. The `OpMode` remains the source of the one telemetry instance used for the run.
2. Telemetry is injected into the Robot Model at construction time, not threaded through the loop as a method argument.
3. Each abstract manual layer has a telemetry-writing responsibility that uses its injected telemetry object.
4. Each abstract autonomous layer also has a telemetry-writing responsibility.
5. Telemetry ownership is consistent across the full hierarchy, not handled one way in Billy and another way elsewhere.
6. The final design defines one clear rule for who is allowed to call `telemetry.update()`.
7. The refactor fixes the current telemetry bugs and failure modes rather than merely moving them around.

Architectural Intent

The desired end state is:

- `OpMode` creates the robot and owns FTC lifecycle.
- `Robot`, `DriveTrain`, `MechAssembly`, and component-level abstractions each receive telemetry as part of construction or controlled initialization.
- Those layers write telemetry through their owned telemetry reference rather than accepting `Telemetry` as a loop-time parameter.
- The autonomous-facing abstractions also expose telemetry-writing entry points.
- Autonomous strategies should be able to report status through the robot model instead of requiring raw telemetry to be passed directly into each strategy.

Scope

At minimum, this assignment applies to the following architectural surfaces:

- Robot
- DriveTrain
- MechAssembly
- MechComponent
- `Robot.AutonomousRobot`
- `DriveTrain.AutonomousDriving`
- `MechAssembly.AutonomousMechBehaviors`
- Any autonomous component surface that needs to participate in the new telemetry model

Concrete implementations in current use must also be updated so the architecture is actually exercised:

- BillyRobot
- Wildbots2025
- MecanumDrive
- StandardTankDrive
- OneStickTank
- BillyMA
- CascadeArm
- ExampleIntakeAssembly
- Relevant mech components and autonomous helpers

Required Deliverables

You must submit all of the following:

1. A design write-up created before major implementation begins.
2. A bug inventory describing the problems caused by the current telemetry design.
3. The completed refactor.

4. A verification report describing what you tested and what behavior you confirmed.
5. A short reflection explaining tradeoffs, compromises, and any follow-up work still needed.

Required Planning Write-Up

Your design write-up must include these sections:

1. Current-State Map

Describe how telemetry currently flows from `OpMode` into the Robot Model and where telemetry is written today.

1. Ownership Rule

State exactly which layer owns the telemetry reference at each point in the hierarchy.

1. Update Rule

State exactly where `telemetry.update()` is allowed to happen and why.

1. Manual-Layer Design

Explain how the manual layers will write telemetry after `Telemetry` is no longer passed as a loop-time argument.

1. Autonomous-Layer Design

Explain how `AutonomousRobot`, `AutonomousDriving`, `AutonomousMechBehaviors`, and any needed autonomous component surfaces will write telemetry.

1. Migration Plan

List the implementation order you will follow and explain why that order reduces breakage.

1. Risk Analysis

Identify the main refactor risks, including constructor churn, partially migrated classes, and behavior regressions.

1. Verification Plan

Describe how you will prove the new design works in both TeleOp and Autonomous.

The write-up must describe architecture and rationale, not just a list of files to edit.

Required Bug Analysis

You must identify the bugs and failure modes caused by the current implementation. Your analysis must include file references, the observed symptom, and the architectural root cause.

At minimum, your write-up must address the following current problems:

1. Duplicate telemetry flushes in the TeleOp loop.

Evidence to inspect:

- Robot
- TeleOp
- BillyTeleOpBLUE
- BillyTeleOpRED

1. Mid-cycle telemetry writes and updates from lower layers, which fragment the telemetry lifecycle.

Evidence to inspect:

- BillyMA
- BillyRapidFire
- LimelightAutoTarget

1. Telemetry hooks that exist in the API but do not actually report data.

Evidence to inspect:

- DoubleShooter
- SpinnyIntake
- StandardTankDrive
- OneStickTank

1. Telemetry strategy objects that are configured but ignored by the implementation.

Evidence to inspect:

- PusherServo
- BillyMA

1. Assemblies that fail to report the telemetry of all relevant child components.

Evidence to inspect:

- BillyMA
- ExampleIntakeAssembly

1. Autonomous code that bypasses the Robot Model and accepts raw telemetry because the autonomous abstractions do not provide their own telemetry path.

Evidence to inspect:

- Robot
- BillyRobot
- BillyRapidFire
- LimelightAutoTarget
- ATagL1Strategy

- ATagL2Strategy

You may identify additional bugs beyond these six. You should.

Constraints

Your refactor must follow these constraints:

- Do not solve this by keeping the same pass-through design and only renaming methods.
- Do not make OpModes thicker.
- Do not introduce a second telemetry object.
- Do not require autonomous strategies to keep accepting raw `Telemetry` just because the base layers are inconvenient.
- Do not silently remove useful telemetry in order to simplify the refactor.
- Do not bury `telemetry.update()` calls all over the hierarchy.

Recommended Execution Steps

Follow this order unless your design write-up justifies a better sequence:

1. Inventory the current telemetry path.

Produce a dependency map showing where telemetry is stored, passed, and flushed today.

1. Define the final telemetry contract.

Decide what each abstract layer is allowed to do with telemetry and what method each layer exposes for writing it.

1. Refactor the abstract manual-layer contracts first.

Change the base model so the architecture itself reflects the new telemetry ownership rule.

1. Refactor the abstract autonomous-layer contracts next.

Add autonomous telemetry-writing responsibilities before touching every concrete strategy.

1. Migrate concrete drive trains, assemblies, and components.

Update implementations so they use owned telemetry rather than loop-time parameters.

1. Fix the known dead and missing telemetry paths.

Do not leave behind components that technically compile but still fail to report.

1. Migrate robot-level helpers and autonomous strategies.

Remove raw telemetry dependencies where the new autonomous surfaces should now cover them.

1. Simplify the OpModes.

The final OpModes should construct the robot, run the loop, and follow the one approved flush rule.

1. Verify both TeleOp and Autonomous behavior.

Confirm that telemetry still appears where expected and that it now follows one consistent lifecycle.

1. Document what changed.

Record which bugs were fixed, what design decisions were made, and what limitations remain.

Acceptance Criteria

The project is not complete until all of the following are true:

- The Robot Model no longer depends on passing `Telemetry` downward every loop as the standard telemetry mechanism.
- The abstract manual layers have a consistent telemetry ownership model.
- The abstract autonomous layers have telemetry-writing methods.
- Concrete robots and subsystems actually use the new design.
- There is one clearly documented rule for when `telemetry.update()` happens.
- Telemetry bugs identified in the bug inventory are fixed or explicitly justified as deferred work.
- TeleOp remains thin.
- Autonomous code can report status without scattering raw `Telemetry` parameters through strategy constructors and helper functions.
- The student can explain why the new design is architecturally better, not just say that it compiles.

Verification Expectations

Your verification report must include:

1. A manual test checklist for at least one TeleOp robot and one Autonomous flow.
2. Evidence that telemetry appears from the drivetrain, mech assembly, and at least one lower-level mechanism.
3. Evidence that autonomous telemetry is visible through the new architecture.
4. Evidence that the final design does not rely on multiple competing telemetry update points in the same cycle.
5. A list of any telemetry that still does not exist and whether that is intentional.

Evaluation Criteria

You will be evaluated on:

- Accuracy of your current-state analysis
- Quality of your bug identification

- Quality of your architectural reasoning
- Discipline of your migration plan
- Completeness of the implementation
- Quality of your verification
- Ability to explain tradeoffs clearly

Final Reflection Prompt

At the end of the project, write a short reflection answering these questions:

1. What was the most important design mistake in the original telemetry architecture?
2. What changed at the abstract-layer level versus the concrete-class level?
3. What telemetry bug was the easiest to miss during code review?
4. What part of the refactor was structurally necessary but not obvious at first?
5. What follow-up refactor would you recommend next?

Definition of Success

Success means the student can defend a coherent answer to this question:

"Why is telemetry now a first-class responsibility of the Robot Model and its autonomous surfaces, instead of a loop-time argument that leaks through the hierarchy?"

If that answer is not clear, the refactor is not finished.